INVENTORY SYSTEM

INSTALLATION MANUAL

Installing, running and backing up the system on the store's office computer

1
Introduction

This manual explains how to install the Inventory System on the store's office computer and how to run it every day. The system runs entirely on that one computer: the database, the server and the website all live on it, and it is opened in a web browser at the address `http://localhost:3000`. No internet connection is needed to use it.

The manual has two kinds of reader:

- **The person installing the system** (sections 2, 3 and 6) sets it up once: installs the required programs, runs the setup tool and signs in for the first time. This person should be comfortable installing Windows programs.
- **The Store Owner and the Secretary** (sections 4, 5 and 6) use the desktop shortcuts to start and stop the system, check it, and make backups. No technical knowledge is needed.

Section 7 is for the developers who build the installation package. How to use each module of the system (orders, inventory, forecasting and so on) is explained in the separate User Manual.

The installation package is a single file named `InventorySystem-<date>.zip`. Table 1.1 lists what it contains.

Table 1.1. Contents of the installation package

Item	Purpose
`Start System.bat`	Starts the system and opens it in the browser.
`Stop System.bat`	Stops the system.
`Restart System.bat`	Stops the system, then starts it again.
`Check System.bat`	Shows whether each part of the system is running.
`Backup Database.bat`	Saves a copy of all data to the *Documents* folder.
`tools\Setup.bat`	One-time setup, run by the person installing.
`tools\Restore Database.bat`	Puts back the data from a backup. For the person installing only.
`backend\`	The server program (`ERP.exe`).

`frontend\`	The website, run by Node.js.
`database\`	`migrate.exe`, which creates the database tables during setup.
`config\`	Settings. Setup writes the computer's own settings here.
`VERSION.txt`	The version of the server and the website, and the build date.
`Installation Manual.pdf`	This manual.

2
System Requirements

Table 2.1. System requirements

Requirement	Minimum
Operating system	Windows 10 or Windows 11, 64-bit
Memory (RAM)	4 GB (8 GB recommended)
Free disk space	3 GB: about 0.5 GB for the system, 1.5 GB for PostgreSQL and Node.js, and room for the data and backups
Database	PostgreSQL 18 (version 16 or newer works), from postgresql.org
Website runtime	Node.js LTS, version 20.9 or newer, from nodejs.org
Browser	A current version of Google Chrome, Microsoft Edge or Mozilla Firefox
Screen	1366 × 768 or larger

PostgreSQL and Node.js are free. Their installers are downloaded once, during installation; after that the system works without the internet.

3
Installation and Setup

These steps are done once, by the person installing the system, while signed in to Windows with an account that can install programs.

3.1 Install PostgreSQL

1. Download the PostgreSQL 18 installer for Windows x86-64 from `https://www.postgresql.org/download/windows/` and run it.
2. Keep the default installation folder (`C:\Program Files\PostgreSQL\18`) and the default components.
3. When asked for a password for the database superuser (`postgres`), choose one and **write it down**. Setup asks for it once.
4. Keep the default port, **5432**, and the default locale.
5. At the end, untick *Launch Stack Builder*; it is not needed.

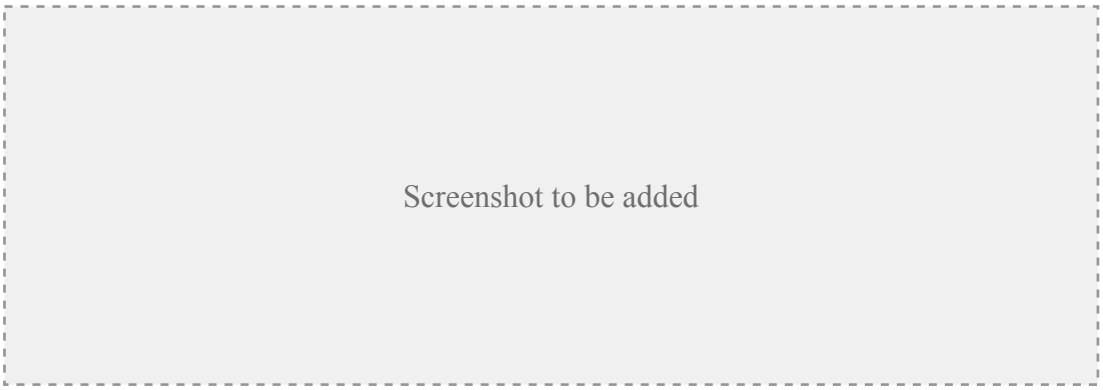


Figure 3.1. PostgreSQL installer: superuser password

3.2 Install Node.js

1. Download the Windows Installer (`.msi`) for the current **LTS** version from `https://nodejs.org` and run it.
2. Keep all the defaults. The option *Automatically install the necessary tools* is not needed and can stay unticked.

3.3 Copy the system to the computer

1. Copy `InventorySystem-<date>.zip` to the computer, for example from a USB drive.
2. Right-click the zip file and choose **Properties**. If there is an **Unblock** box at the bottom of the *General* tab, tick it and press **OK**. This stops Windows from showing a security warning every time a shortcut is opened.
3. Right-click the zip file, choose **Extract All…**, and extract it to `C:\`.
4. Rename the extracted folder `C:\InventorySystem-<date>` to `C:\InventorySystem`.

The folder can be anywhere on the computer, but the rest of this manual uses `C:\InventorySystem`. Do not move the folder after setup; if it is moved, run setup again.

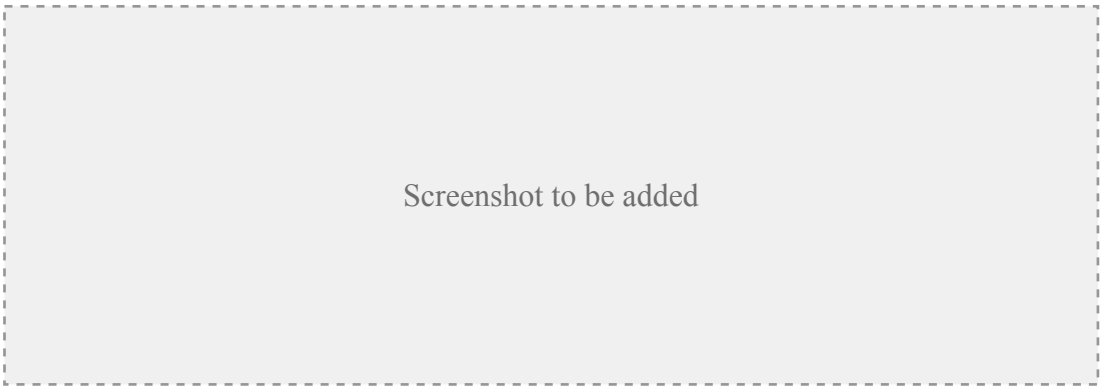


Figure 3.2. The `C:\InventorySystem` folder after extracting

3.4 Run the setup tool

1. Open `C:\InventorySystem\tools` and double-click **Setup.bat**.
2. Setup checks that PostgreSQL and Node.js are installed. If one is missing, it says so and stops; install it and run setup again.
3. When asked, type the `postgres` password from Section 3.1 and press **Enter**. Nothing appears on the screen while typing; this is normal. A wrong password can be retried three times.

4. Setup then works on its own. It creates the system's own database user (`erp`) with a new random password, creates the database named `ERP`, writes the settings, creates the tables with the system's default data, and puts five shortcuts on the desktop: **Start System**, **Stop System**, **Restart System**, **Check System** and **Backup Database**.
5. When it shows *Setup finished*, press **Y** to start the system.

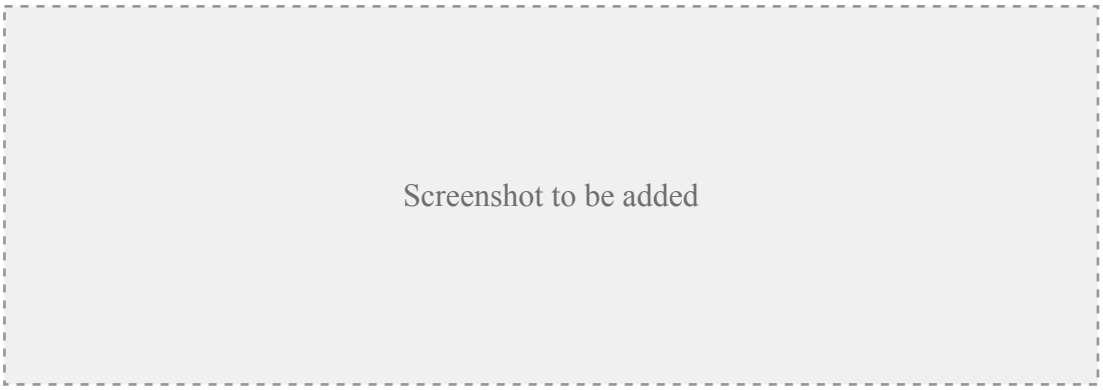


Figure 3.3. Setup.bat after a successful setup

Setup creates no sample products, orders or sales. The database starts with only what the system needs to work: the roles, the order and stock statuses, the default units of measurement, the standard discounts (such as the senior citizen and PWD discounts), and two user accounts (Table 3.1).

Setup can be run again at any time, for example after the folder was moved. It keeps all the data. It gives the system a new database password and a new sign-in key, so everyone has to sign in again afterwards.

3.5 First sign-in

The browser opens the system at `http://localhost:3000`. Sign in with the Owner account from Table 3.1.

Table 3.1. Default accounts

Account	Email	Password	Role
System Owner	`owner@inventory.local`	`mypassword`	Owner

System Secretary	`secretary@inventory.local`	`mypassword`	Secretary

Everyone who has this manual knows these passwords, so change them right away:

1. Signed in as the Owner, open **Account Settings** (from the profile button at the bottom of the sidebar) and change the Owner's password. The name and email can also be changed here, for example to the owner's real name.
2. Sign out, sign in as the Secretary, and change the Secretary's password the same way. Then give the new password to the Secretary.
3. Before adding any data, make a first backup with **Backup Database** (Section 6.1).

The Owner account (the first account) always stays an Owner; its role cannot be changed. More accounts can be added in the **Accounts** module, as explained in the User Manual.

4
Running the System Day to Day

The Owner and the Secretary run the system with the shortcuts on the desktop.

4.1 Starting the system

1. Double-click **Start System** on the desktop.
2. A window shows each part starting: *Database*, *Server* and *Website*. The first start after turning on the computer can take up to a minute.
3. When all three show **OK**, the system opens in the browser by itself. The window closes after a few seconds.
4. Sign in with your email and password.

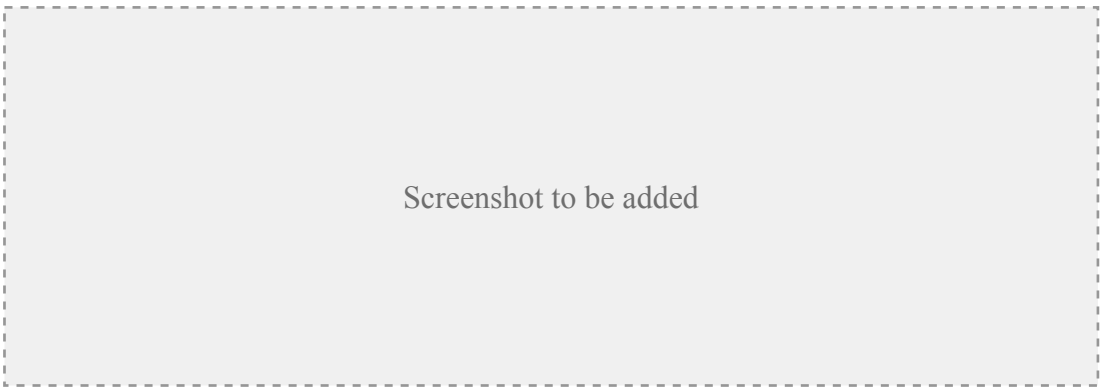


Figure 4.1. Start System with the Database, Server and Website all OK

Two small windows named *Inventory System - Server* and *Inventory System - Website* appear minimized on the taskbar. **Do not close them**: they are the system. Closing one stops that part of the system.

Starting the system again while it is already running does no harm: it only opens the browser. A bookmark to `http://localhost:3000` also works while the system is running.

4.2 Stopping the system

Double-click **Stop System** when the store closes, or simply shut down the computer; both are safe. Data is saved the moment each record is saved in the system, so nothing is lost by stopping.

The database itself keeps running in the background. It starts by itself with Windows and uses little memory.

4.3 Restarting and checking

- **Restart System** stops and starts the system. Use it first whenever the system seems stuck.
- **Check System** shows, in plain words, whether the *Database*, the *Server* and the *Website* are running (**OK** or **NOT RUNNING**), what to do if one is not, and the system's version.

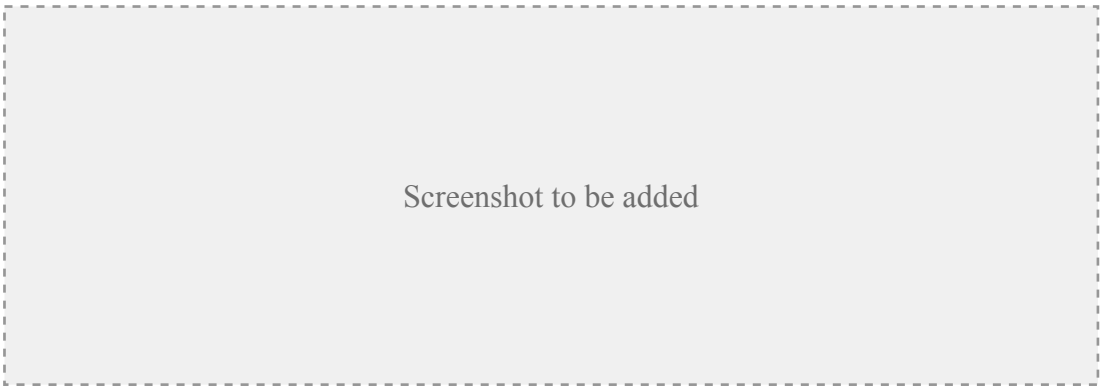


Figure 4.2. Check System when everything is running

5
Troubleshooting

Most problems are solved by **Restart System**. When that does not help, Table 5.1 lists what to do. The server and website write their messages to `C:\InventorySystem\logs\server.log` and `website.log`; send these files to the person who installed the system when asking for help.

Table 5.1. Troubleshooting

What you see	What to do
The browser says the page can't be reached (`localhost:3000`)	The system is not running. Double-click **Start System**.
Something went wrong in the system	The website cannot reach the server. Double-click **Restart System**, then sign in again.
Session Expired	You were signed out, for example after a long time or after setup was run again. Press **Return to Login** and sign in.
Start System says the *Database* did not start	Restart the computer and try again. If it still fails, the person installing should check that the *postgresql-x64-18* service is running and set to *Automatic* in Windows *Services*.
Start System says the *Server* or *Website* did not start within the time limit	Double-click **Restart System**. If it fails again, restart the computer. If it still fails, send the log file named in the message to the person who installed the system.
A part shows **NOT RUNNING** in Check System	Double-click **Restart System**.
The server or website does not start because port 5176 or 3000 is already in use	Another program uses the same port. Restart the computer. If it keeps happening, the other program must be closed or uninstalled; ask the person who installed the system.
The shortcuts are missing from the desktop	Open `C:\InventorySystem` and double-click the `.bat` file with the same name, or run `tools\Setup.bat` again to recreate the shortcuts.
Windows asks whether to allow `ERP.exe` or Node.js through the firewall	Press **Cancel**. The system is only used on this computer and needs no network access.
Forgotten password	The system has no "forgot password" option, and the Owner cannot see or change another user's password. Ask the person who installed the system to reset it (Section 5.1).

5.1 Resetting a forgotten password

This is for the person who installed the system. It sets one account's password back to `mypassword`; the user then signs in and changes it in **Account Settings**.

1. Open the Windows *Command Prompt* (search for `cmd` in the Start menu).
2. Type the two commands below, replacing `owner@inventory.local` with the email of the account, and press **Enter** after each.

```
call C:\InventorySystem\config\system.cmd
"%PGBIN%\psql.exe" -c "UPDATE \"UserAccounts\" SET \"Salt\" = '0mUBGSIov6YDrqLo4SaFuw==',
\"Password\" = '0ULmzYhoKo1MJ9WyeS3WAktEqN/QYeXPEb7z7akCUxM=' WHERE \"Email\" =
'owner@inventory.local';"
```

The command should answer `UPDATE 1`. `UPDATE 0` means no account has that email.

6
Backup and Restore

6.1 Making a backup

All of the store's data (products, stock, orders, sales, purchases, accounts and audit logs) is in the database on this one computer. If the computer breaks or its disk fails, the data is lost unless there is a backup.

1. Double-click **Backup Database** on the desktop. The system can be running or stopped.
2. The window shows where the backup was saved, and a folder window opens with the new file selected. Backups are saved in the *Documents* folder, in *Inventory Backups*, with the date and time in the name, for example `ERP-20261021-173000.backup`.
3. Copy the newest backup to a USB drive or to cloud storage regularly, so a copy survives if the computer is lost.

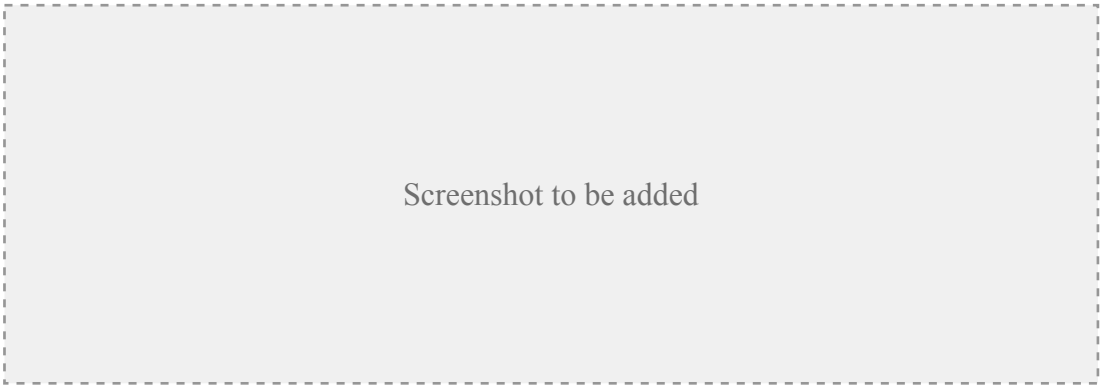


Figure 6.1. Backup Database after a successful backup

Make a backup at least **once a day** at closing time, and always before running setup again or restoring a backup. Old backups can be deleted once there are newer ones saved elsewhere; a backup is usually small.

6.2 Restoring a backup

Restoring **replaces all current data** with the data in the backup, so anything recorded after that backup was made is gone. This is why it is in the `tools` folder and not on the desktop: only the person who installed the system should do it, for example after the database was damaged or the system was installed on a new computer.

1. Open `C:\InventorySystem\tools` and double-click **Restore Database.bat**. (A `.backup` file can also be dragged onto it.)
2. Choose the backup file. The window opens in the *Inventory Backups* folder.
3. Read the warning and type `YES` in capital letters to continue. Anything else cancels and changes nothing.
4. The tool stops the system, saves the current data as `ERP-before-restore-<date>.backup` in the same folder, restores the chosen backup, and starts the system again.

If the restore fails, the message names the `before-restore` backup; restoring that file puts the data back as it was.

To **move the system to a new computer**: make a backup on the old computer, install the system on the new one (Section 3), then restore the backup there. The accounts and passwords from the backup apply after the restore.

7
For Developers: Building the Release Package

The release package is built on the developer's Linux machine by one script, `deploy/package.sh`, in the project folder next to `Backend/` and `Frontend/`. It builds a Windows package; it does not run anything on Windows.

7.1 Prerequisites

- The .NET SDK (9 or newer) and the Entity Framework tool: `dotnet tool install --global dotnet-ef`.
- Node.js and npm, with the Frontend's packages installed (`npm install` in `Frontend/`).
- `git` and `zip`.
- Internet access the first time, to download the Windows runtime packs for .NET.

7.2 Building

1. Commit all changes in `Backend/` and `Frontend/`. The script refuses to package uncommitted changes unless it is run with `--allow-dirty`, so the package always matches a version in git.
2. Run `./deploy/package.sh` from the project folder. To change how hard the zip is compressed, add `--compress=max` for the smallest file (slower) or `--compress=none` to store the files uncompressed (fastest, biggest file). Without it, zip's default level is used.
3. The result is `deploy/dist/InventorySystem-<YYYY.MM.DD>.zip`.

   The script:

- exports the committed Backend with `git archive` and publishes it with `dotnet publish -c Release -r win-x64 --self-contained`, so the office PC needs no .NET installed;
- builds `migrate.exe` with `dotnet ef migrations bundle --target-runtime win-x64 --self-contained`; setup runs it to create the tables;
- builds the Frontend with `npm run build` (`output: "standalone"` in `next.config.ts`) and copies `.next/standalone`, `.next/static` and `public`; Node.js on the office PC runs `server.js`;
- copies the Windows scripts from `deploy/windows/` with Windows (CRLF) line endings, and writes `VERSION.txt`;
- copies `deploy/docs/installation-manual.pdf` as `Installation Manual.pdf`, so run `typst compile docs/installation-manual.typ` and commit the PDF after editing this manual (the script stops if the PDF is older than its `.typ` sources);
- checks the result and stops with an error if a required file is missing or a secret leaked.

7.3 What never goes into the package

The developer's `Backend/appsettings.json` holds the development database password and sign-in (JWT) key, and `Frontend/.env` holds development settings, including one that turns off HTTPS certificate checks. None of them is shipped:

- The Backend is built with `deploy/config/appsettings.template.json` in place of `appsettings.json`, and the generated `appsettings.json` is removed from the output. On the office PC, `Setup.bat` writes

`backend\appsettings.json` from the template with a new random database password and JWT key.

- The `.env` file that Next.js copies into `.next/standalone` is deleted. `Start System.bat` sets the website's settings (`PORT`, `HOSTNAME`, `BACKEND_API_URL`) itself.
- Before zipping, the script searches the whole package for the development JWT key and database password and stops if either is found.

7.4 Versions

The Backend version comes from `<Version>` in `ERP.csproj` and the Frontend version from `package.json`, both following Semantic Versioning and the `CHANGELOG.md` of each repository. `VERSION.txt` in the package records both and the build date, and **Check System** shows it on the office PC.

7.5 Production settings

Table 7.1. Production settings

Setting	Value on the office PC
Server address	`http://localhost:5176` (`Urls` in `appsettings.json`); production mode, so Swagger is off
Website address	`http://127.0.0.1:3000`, opened as `http://localhost:3000`
Database	PostgreSQL on `localhost:5432`, database `ERP`, owned by the role `erp`
Shared script settings	`config\system.cmd`, written by setup: PostgreSQL folder and service, database login, Node.js path
Logs	`logs\server.log` and `logs\website.log`; the previous run's logs are kept as `*.old.log`

The scripts were written on Linux. Before this manual is final, the whole procedure in sections 3 to 6 must be done once on a real Windows 10 or 11 computer, and the screenshots taken there.